Cap. 3: Programación Lineal

2 de noviembre de 2026

Esquema de la presentación

¿Qué es un programa lineal?

Polítopos, vértices y holgura

El método simplex de Dantzig

El método del tableau

Dualidad

Programación entera

Cierre

El problema central

- Hasta aquí: optimización *sin restricciones*.
- La **programación lineal (PL)**: problemas *convexos con restricciones*, con función objetivo *lineal*.
- Nace de recursos limitados (tiempo, espacio, dinero, materiales), que producen *tradeoffs* expresados como desigualdades.
- Se desarrolló en la Segunda Guerra Mundial (racionamiento y logística).
- Sin solución analítica en forma cerrada, pero con algoritmos exactos que escalan a miles de variables y restricciones.

Forma estándar

$$\max_x \; c^T x \quad \text{s.t.} \quad Ax \leq b, \qquad x \geq 0$$

$c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Brunton, *Optimization Bootcamp*, Cap. 3

La geometría lo determina todo

- Cada restricción $a_j^T x \leq b_j$ es un **semi-espacio**; el conjunto factible es la intersección de semiplanos con el cuadrante positivo: un **polítopo convexo**.
- El objetivo lineal es a la vez convexo y cóncavo: tanto maximizar como minimizar son tareas **convexas**.
- El gradiente del objetivo es *constante*: $\nabla f(x) = c$.
- **El óptimo siempre ocurre en un vértice** del politopo.
- **Óptimo local = óptimo global**: al ser convexo, basta encontrar un extremo local.

El problema combinatorio

Un hipercubo de dimensión n tiene 2^n vértices: 2^{20} supera el millón. Enumerar vértices es intratable $\Rightarrow$ hace falta un algoritmo de búsqueda eficiente: el **simplex**.

Brunton, *Optimization Bootcamp*, Cap. 3

Ejemplo: la fábrica de microscopios

Ganancia \$100 (enseñanza) y \$400 (investigación); capacidad total 50/mes; demanda máxima de investigación 30/mes.

$$\max_{x_1,x_2} \; 100x_1 + 400x_2 \quad \text{s.t.} \quad x_1 + x_2 \le 50, \;\; x_2 \le 30, \;\; x_1, x_2 \ge 0$$

Vértice	(x_1, x_2)	f
v_1	$(0, 0)$	0
v_2	$(50, 0)$	5000
v_3	$(0, 30)$	12000
v_4	$(20, 30)$	**14000**

En Python

`pulp`: `LpMaximize`, `LpVariable`, `solve()`. Da $x_1 = 20$, $x_2 = 30$, profit 14000.

Brunton, *Optimization Bootcamp*, Cap. 3

Geometría de los politopos

- Cada frontera del politopo la define una desigualdad **activa** (*tight*), que se comporta como una igualdad.
- Cada **vértice** es la intersección de n fronteras activas en n dimensiones.
- Para acotar un politopo en n dimensiones hacen falta al menos $n+1$ superficies $(n-1)$-dimensionales.
- Vértices adyacentes están unidos por una arista unidimensional.

Variables de holgura

$$\max_x \; c^T x \quad \text{s.t.} \quad Ax + s = b, \qquad x, s \geq 0$$

s_i cuantifica la distancia a la frontera i: $s_i = 0 \Rightarrow$ **la restricción i está activa.** Un punto interior tiene todas las holguras estrictamente positivas.

Sinónimos en la literatura: variables *tight* = no básicas; variables *slack* = básicas.

Brunton, *Optimization Bootcamp*, Cap. 3

Por qué importa la holgura

Encontrar un vértice (desde cero)

- n restricciones activas $\Rightarrow$ sistema lineal $\tilde{A}x = \tilde{b}$.
- Se resuelve por inversión de matriz: coste $\mathcal{O}(n^3)$.
- Prohibitivo en dimensiones altas.

Caminar a un vértice vecino

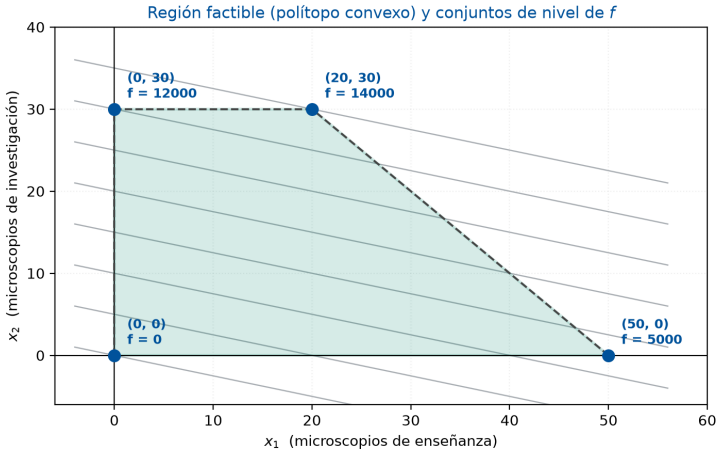
- Basta *tensar* una holgura y *aflojar* una tight.
- Se intercambia un solo par de variables.
- Mucho más barato: es la **base del simplex**.

Idea clave

No se resuelven vértices: se camina de vértice en vértice, cambiando una restricción activa por una inactiva en cada paso.

Brunton, *Optimization Bootcamp*, Cap. 3

La región factible y sus vértices



Los conjuntos de nivel de f son rectas paralelas (el gradiente es constante); el máximo y el mínimo tocan siempre algún vértice.

Brunton, *Optimization Bootcamp*, Cap. 3

La idea

- Dos propiedades bastan:
  1. un extremo local ocurre en un vértice;
  2. el extremo local es global (convexidad).
- Se **camina de vértice en vértice** (*pivoting*) por aristas que aumentan el objetivo, hasta que ninguna dirección libre mejora.
- El objetivo crece **monótonamente** de pivote en pivote.
- Cualquier variable holgada puede aumentar sin violar nada: su holgura es no nula.

Brunton, *Optimization Bootcamp*, Cap. 3

Regla de pivoteo de Dantzig

1. **Aflojar una restricción activa (elegir la columna).** Se elige la dirección en que el objetivo más crece: la componente mayor del gradiente $\nabla f = c$. Esa variable tight se afloja.
2. **Tensar una variable holgada (elegir la fila).** Entre las holguras candidatas se toma la que se violaría *primero* al aumentar la variable recién aflojada: la de **menor razón** b_j / a_j.
3. **Recentrar** en el nuevo vértice: se intercambian las posiciones de la variable aflojada y la tensada, y se sustituye la aflojada en el objetivo y en las demás restricciones.

Es casi idéntico a la eliminación gaussiana con pivoteo.

Brunton, *Optimization Bootcamp*, Cap. 3

El ejemplo a mano

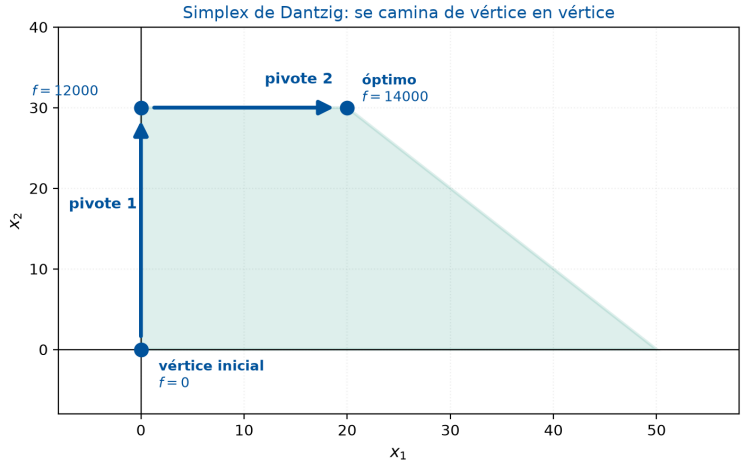
Forma con holgura (microscopios):

$$s_1 = 50 - x_1 - x_2, \qquad s_2 = 30 - x_2$$

- **Vértice** $(0, 0)$: x_1, x_2 tight; $s_1 = 50$, $s_2 = 30$ (holgadas). El gradiente $(100, 400)$ es mayor en $x_2 \Rightarrow$ se afloja x_2.
- Al aumentar x_2, la restricción $s_2 = 0$ se viola primero $(30/1 < 50/1) \Rightarrow$ se tensa s_2, y $x_2 = 30 - s_2$.
- Sustituyendo, el objetivo pasa a máx $100x_1 - 400s_2 + 12000$: nuevo vértice $(0, 30)$, $f = 12000$.
- Segundo pivote: se afloja x_1 y se tensa $s_1 \Rightarrow$ vértice $(20, 30)$, $f = 14000$. **Óptimo.**

Brunton, *Optimization Bootcamp*, Cap. 3

El camino de vértice en vértice



Cada pivote recorre una arista del politopo; el objetivo nunca disminuye.

Brunton, *Optimization Bootcamp*, Cap. 3

Tableau simplex

Reescribir el PL entero en cada pivote es engorroso. El *tableau* lo hace sistemático, con operaciones de fila tipo eliminación gaussiana:

	f	x	s	
f	1	$-c^T$	0	0
s	0	A	I	b

- La **fila objetivo** codifica f en términos de las variables activas.
- La **fila de restricciones** codifica $Ax + s = b$.
- La última columna lleva el valor actual de f y de cada s_i.
- Es la misma operatoria que invertir una matriz: $\left[\, A \mid I \,\right] \to \left[\, I \mid A^{-1} \,\right]$.

Brunton, *Optimization Bootcamp*, Cap. 3

Iteración de pivoteo

1. **Columna a aflojar:** la variable activa con el coeficiente más negativo en la fila objetivo (en un problema de maximización).
2. **Fila a tensar:** test de razón mínima, $\mathrm{RHS}/\text{coef. pivote}$ más pequeño.
3. **Reducción por filas** alrededor del pivote: la entrada pivote se vuelve 1 y el resto de la columna, 0.

Criterio de paro

Se itera hasta que **no quedan coeficientes negativos en la fila objetivo**. Ese vértice es óptimo local y, por convexidad, **global**.

Brunton, *Optimization Bootcamp*, Cap. 3

Ejemplo: el tableau paso a paso

	f	x_1	x_2	s_1	s_2	RHS
f	1	-100	-400	0	0	0
s_1	0	1	1	1	0	50
s_2	0	0	1	0	1	30
tras el pivote en x_2 (se afloja x_2, se tensa s_2)						
f	1	-100	0	0	400	12000
s_1	0	1	0	1	-1	20
x_2	0	0	1	0	1	30
tras el pivote en x_1 (se afloja x_1, se tensa s_1)						
f	1	0	0	100	300	**14000**
x_1	0	1	0	1	-1	20
x_2	0	0	1	0	1	30

Coincide exactamente con el método a mano de la sección anterior: vértice $(20, 30)$, $f = 14000$. Ya no hay coeficientes negativos $\Rightarrow$ óptimo.

Brunton, *Optimization Bootcamp*, Cap. 3

Excepciones y casos borde

- **Problema no acotado**: si en el test de razón mínima todos los coeficientes de pivote son negativos, el objetivo crece sin límite. Ej.: máx $x_1 + x_2$ s.t. $2x_1 - x_2 \geq 2$, $2x_2 - x_1 \geq 2$ (el dual es infactible).
- **Degeneración**: varias restricciones activas a la vez o empate en el test de razón $\Rightarrow$ **ciclado**. Se rompe con la **regla de Bland**.
- **Forma no estándar**: igualdades $Ax = b$ o restricciones $Ax \geq b$ no admiten holgura directa; el simplex necesita partir de una *solución básica factible*.

Phase I y Phase II

Se introducen **variables artificiales** $r \geq 0$ y se resuelve mín $1^T r$ con $Ax + r = b$. Si el mínimo es 0, hay solución factible y se pasa a **Phase II**; si es > 0, el problema es **infactible**.

Brunton, *Optimization Bootcamp*, Cap. 3

El problema dual

A todo programa lineal (*primal*) le corresponde otro (*dual*) que acota su valor óptimo. La relación es una **transposición**:

$$\underbrace{\text{máx } c^T x \quad \text{s.t.} \quad Ax \leq b,\ x \geq 0}_{\text{primal}} \qquad \Longrightarrow \qquad \underbrace{\text{mín } b^T y \quad \text{s.t.} \quad A^T y \geq c,\ y \geq 0}_{\text{dual}}$$

- c del primal $\to$ b del dual; $b \to c$; $A \to A^T$.
- Se interpreta *multiplicando* las restricciones del primal por y_1, y_2, y_3 y sumando, para acotar $c^T x$ por arriba.

Brunton, *Optimization Bootcamp*, Cap. 3

Dualidad débil, fuerte y su gap

Dualidad débil

Para todo x factible del primal y todo y factible del dual:

$$c^T x \leq b^T y \quad \Longrightarrow \quad \max_{x \in X} c^T x \ \leq \ \min_{y \in Y} b^T y.$$

La brecha entre ambos óptimos es el **duality gap**.

Dualidad fuerte

Si el primal tiene solución básica factible óptima, los dos valores **coinciden** y el gap es cero. Cada solución factible de un problema *certifica* la optimalidad del otro.

- y_j = multiplicador de Lagrange de la restricción j: **precio sombra** o **valor marginal**.
- Mide la sensibilidad del valor óptimo ante cambios en b y en c.
- Los métodos de punto interior resuelven primal y dual simultáneamente.

Brunton, *Optimization Bootcamp*, Cap. 3

Programación lineal entera (ILP)

$$\max_x \; c^T x \quad \text{s.t.} \quad Ax \leq b, \qquad x \in \{0,1\}^n \text{ o } \mathbb{N}^n$$

- Variables enteras: decisiones de sí/no (empaque, ubicación, inclusion).
- Muchos ILP son **NP-hard**: no se conoce algoritmo polinomial en el peor caso.
- **Relajación LP**: se permiten x continuas ≥ 0. Toda solución entera factible es factible para la relajación, luego la relajación da una **cota superior** del óptimo entero.
- Si el óptimo de la relajación ya es entero, *es* el óptimo del ILP.
- En el ejemplo de los microscopios los vértices eran enteros: PL e ILP coinciden.

Brunton, *Optimization Bootcamp*, Cap. 3

Branch-and-bound y planos de corte

Branch-and-bound

- Se resuelve el PL relajado.
- Para cada x_j no entera se crea un subproblema con $x_j \leq \lfloor x_j \rfloor$ y otro con $x_j \geq \lceil x_j \rceil$.
- Si la cota de un subproblema es peor que una solución entera conocida, esa rama se **poda**.
- Construye un árbol de búsqueda.

Planos de corte

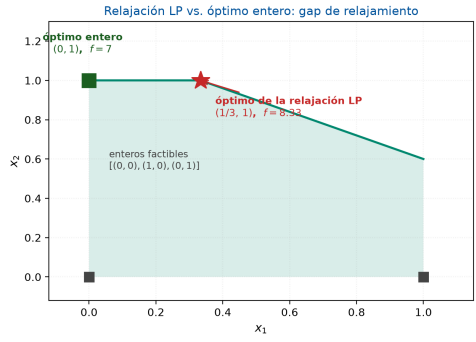
- Se agregan restricciones lineales (*cortes*) que eliminan soluciones no enteras y conservan todos los enteros factibles.
- Cada corte estrecha la región factible hacia el *casco entero*.
- Tipos: de Gomory, *cover cuts*, desigualdades de cliques.

En la práctica

La mayoría de los solvers usa un enfoque híbrido **branch-and-cut**: la estructura de búsqueda global del B&B con cortes que reducen la ramificación.

Brunton, *Optimization Bootcamp*, Cap. 3

Relajación LP frente al óptimo entero



El óptimo de la relajación es una cota superior del entero; si es fraccionario, no se puede aceptar como solución.

Brunton, *Optimization Bootcamp*, Cap. 3

Dos problemas clásicos

Problema de la mochila (knapsack) — NP-hard:

$$\max_{x_j \in \{0,1\}} \sum_{j=1}^{n} p_j x_j \quad \text{s.t.} \quad \sum_{j=1}^{n} w_j x_j \leq W$$

- Prototipo de la asignación de recursos: empaque de cajas, calendarización de tareas, reparto de presupuesto.
- 2^n subconjuntos: la enumeración es exponencial.
- Panel 3.3 del libro: con `pulp` y con fuerza bruta se obtiene la misma solución óptima (ítems 1, 2, 5, 6; peso 20; valor 81).

Agente viajero (TSP) — NP-hard:

$$\min_{x_{ij} \in \{0,1\}} \sum_{(i,j) \in E} c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} = 1 \ \forall i, \quad \sum_i x_{ij} = 1 \ \forall j$$

- Una sola ruta que visite cada ciudad exactamente una vez y regrese al inicio.
- Requiere restricciones adicionales para eliminar los *sub-tours*.
- Se combina con geometría: empaque en planta, layout de almacén, **diseño de chips VLSI**.

Brunton, *Optimization Bootcamp*, Cap. 3

Puntos para recordar

1. La PL es convexa: el óptimo está en un vértice y es global.
2. Las **variables de holgura** traducen .ªctiva/inactiva.ª "$s_i = 0$ / $s_i > 0$": esa notación es la que hace posible el simplex.
3. El simplex **camina de vértice en vértice**; el tableau es su implementación matricial (pivoteo tipo gaussiana).
4. Excepciones a vigilar: no acotado, degeneración (ciclado) y forma no estándar (Phase I).
5. La **dualidad** da cotas y certificados de optimalidad; y son precios sombra.
6. La programación entera es NP-hard; la relajación LP acota y el branch-and-cut resuelve.

Idea final

La PL es notable porque combina una geometría limpia (politopos, vértices) con un algoritmo exacto y enumerable, y sirve de banco de pruebas para dualidad, optimalidad y enteros.

Para profundizar

Textos

- Dantzig, *Linear Programming and Extensions*, 1963.
- Bertsimas y Tsitsiklis, *Introduction to Linear Optimization*, 1997.
- Wolsey y Nemhauser, *Integer and Combinatorial Optimization*, 2014.
- Schrijver, *Theory of Linear and Integer Programming*, 1998.
- Hillier y Lieberman, *Introduction to Operations Research*, 2021.
- Korte y Vygen, *Combinatorial Optimization*, 2018.
- Martello y Toth, *Knapsack Problems*, 1990.

Artículos

- Dantzig, *Maximization of a linear function of variables subject to linear inequalities*, 1951.
- Karmarkar, *A new polynomial-time algorithm for linear programming*, 1984.
- Gomory, *Outline of an algorithm for integer solutions to linear programs*, 1960.

¿Preguntas?